Multi-Modal Person Re-Identification and Face Association Pipeline

Complete Technical Report

Project	Real-time person re-identification and face association for retail analytics
Environment	Kaggle (2× NVIDIA Tesla T4 GPUs, 16 GB VRAM each)
Language	Python 3.12
Framework	PyTorch

Abstract

This report documents a real-time, multi-modal pipeline for person re-identification (Re-ID) and face association in a retail surveillance setting. The system fuses body appearance embeddings (CLIP-ReID, 1280-dimensional) with face identity embeddings (InsightFace ArcFace, 512-dimensional) to assign persistent global identifiers to individuals across camera views and across sessions. Key design decisions include an NMS-free transformer detector (RT-DETR), a skeleton-based quality gate (YOLOv11s-Pose), temperature-scaled softmax identity resolution, DBSCAN-based unknown clustering, and face-driven retroactive identity merging. Cloud components include Pinecone vector indices, MongoDB Atlas, and ImageKit CDN. Evaluation on 1,317 body crops achieves an accuracy of 85.35% at an average query latency of 183.8 ms.

Contents

1 System Overview **3**
1.1 Functional Description . . . 3
1.2 Dual-Modality Design . . . 3
1.3 Pipeline Architecture . . . 3

2 AI Models and Specifications **4**
2.1 RT-DETR-L — Person Detection . . . 4
2.2 YOLOv11s-Pose — Skeleton Quality Gate . . . 4
2.3 BotSort — Multi-Object Tracker . . . 5
2.4 CLIP-ReID — Body Re-Identification . . . 5
2.5 InsightFace SCRFD — Face Detection . . . 5
2.6 InsightFace ArcFace — Face Recognition . . . 5
2.7 GPU Distribution . . . 6

3 Cloud Infrastructure **6**
3.1 Pinecone — Vector Database . . . 6
3.2 MongoDB Atlas . . . 7
3.3 ImageKit — CDN . . . 7

4 Configuration and Thresholds **7**
4.1 Detection and Quality . . . 8
4.2 Identity Resolution . . . 8
4.3 Face Recognition . . . 8
4.4 Unknown Data Manager (DBSCAN) . . . 8
4.5 Session Control . . . 8

5 Module 4: Video Processing Pipeline **8**
5.1 Gallery Hydration (Pre-Processing) . . . 8
5.2 Frame Processing Loop . . . 8
5.3 Self-Adaptive Gallery (`GalleryManager`) . . . 11
5.4 Identity Merge (`execute_identity_merge`) . . . 11
5.5 Sighting Tracking . . . 12
5.6 Cloud Upload (Post-Processing) . . . 12

6 Module 5: Search Engine **12**
6.1 Body Search with α-FPS Prototypes . . . 12
6.2 Face Search (Direct) . . . 13
6.3 Movement Heatmaps . . . 13

7 Module 6: Interactive Dashboard **14**

8 Evaluation Results **14**
8.1 Body Search — Brute-Force Test 14
8.2 Face Search 15

9 Known Limitations and Design Decisions **15**
9.1 Data Consistency After Reset 15
9.2 RAM-First Processing 15
9.3 Single-Frame Identity Lock 15
9.4 Face Camera Toggle 15
9.5 Embedding Dimension Mismatch 15
9.6 Heatmap Accuracy Depends on Bounding Box Storage 15
9.7 Camera Background Registration 15

1. System Overview

1.1 Functional Description

The pipeline continuously processes security camera footage from a supermarket and performs the following operations on every frame:

1. **Detection.** Every person visible in the frame is localised with a bounding box.
2. **Tracking.** Each detection is linked to a short-lived `track_id` that persists while the person remains continuously visible.
3. **Identification.** Body appearance and/or face embeddings are matched against a persistent database to yield a permanent `global_id`.
4. **Recording.** Visit metadata is stored: entry and exit timestamps, dwell time, best body and face crops.
5. **Search.** Given a query image, the system retrieves the matching person from the database.

1.2 Dual-Modality Design

The system operates with two complementary biometric modalities, whose properties are summarised in Table 1.

Table 1: Comparison of the two identification modalities.

Modality	Strength	Weakness	Role
Body Re-ID	Works at any distance and angle; does not require face visibility	Fails with similar clothing (e.g. store uniforms)	Primary identification
Face Recognition	Extremely accurate; provides identity ground truth	Requires a close-up, front-facing, well-lit face	Override & correction

The key design insight is that face recognition serves as *ground truth*. When a face is positively recognised, it can retroactively correct an erroneous body-based assignment through an *Identity Merge* operation.

1.3 Pipeline Architecture

The high-level data flow is as follows:

```
Video Frame
    |
    +---> RT-DETR (Person Detection)
    |          |
    |          v
    |     BotSort (Multi-Object Tracking) --> track_id (temporary)
    |          |
    |          v
    |     YOLOv11s-Pose (Quality Gate) --> reject bad crops
    |          |
    |          v
    |     CLIP-ReID (Body Embedding) --> 1280-D vector
    |          |
    |          v
    |   +-----------------------------------------------+
```

```
|   |        IDENTITY RESOLUTION ENGINE            |
|   |                                              |
|   |  Gallery Match  --> Softmax Confidence       |
|   |  Face Override  --> Pinecone Face Lookup     |
|   |  Unknown Pool   --> DBSCAN Clustering        |
|   |                                              |
|   |  Output: global_id (permanent)               |
|   +----------------------------------------------+
|
+---> InsightFace (Face Detection + Embedding)
          |
          v
      Face-Body Spatial Pairing --> 512-D face vector
```

Listing 1: High-level pipeline data flow.

2. AI Models and Specifications

2.1 RT-DETR-L — Person Detection

RT-DETR (Real-Time DEtection TRansformer, Large variant) is used as the primary person detector. Key properties are listed in Table 2.

Table 2: RT-DETR-L model specifications.

Property	**Value**
Architecture	CNN backbone (HGNet-v2) + Efficient Hybrid Encoder + Transformer decoder
Parameters	≈32–36 million
FLOPs	≈100–140 GFLOPs
Model size	≈130 MB
Input resolution	640×640
VRAM (inference)	≈2–4 GB
GPU assignment	`cuda:0`

RT-DETR is preferred over YOLO-family detectors because its transformer decoder eliminates Non-Maximum Suppression (NMS), which is a serial post-processing bottleneck. This produces more predictable latency, which is critical for real-time tracking. The detector is restricted to COCO class 0 (person) via `classes=[0]`.

2.2 YOLOv11s-Pose — Skeleton Quality Gate

Table 3: YOLOv11s-Pose model specifications.

Property	**Value**
Architecture	YOLOv11 Small with Pose head
Keypoints	17 (COCO format)
Input	Cropped body image
GPU assignment	`cuda:1`

This model does not contribute to identification. It functions purely as a *quality gate*. For each

body crop the system computes:

$$\text{visible_ratio} = \frac{\text{keypoints with confidence} > 0.3}{17}$$

If visible_ratio < 0.5, the crop is rejected, preventing partial or occluded detections from corrupting identity embeddings.

2.3 BotSort — Multi-Object Tracker

Table 4: BotSort configuration.

Property	**Value**
Full name	Bag of Tricks for SORT
Type	Tracking-by-detection
ReID backbone	OSNet_x0_25 (MSMT17 pretrained)
Association	IoU + ReID appearance fusion

BotSort operates as follows. It receives person detections each frame, applies a Kalman Filter to predict where existing tracks should be, and uses Camera Motion Compensation when the camera moves. A cost matrix combining IoU (spatial overlap) and appearance similarity (via OSNet) is optimised with the Hungarian algorithm to match detections to tracks. The resulting `track_id` persists only while the person is continuously visible; re-entry yields a new `track_id`. The Identity Resolution Engine is responsible for mapping `track_id` to the persistent `global_id`.

2.4 CLIP-ReID — Body Re-Identification

Table 5: CLIP-ReID specifications.

Property	**Value**
Architecture	CLIP ViT-B/16 fine-tuned for ReID
Training dataset	Market-1501
Parameters	≈86 million
Embedding dimension	1280-D
Training strategy	2-stage: (1) optimise learnable text tokens; (2) fine-tune visual encoder
GPU assignment	`cuda:1`

CLIP-ReID maps a cropped body image to a 1280-dimensional vector encoding appearance (clothing colour, texture, body shape, accessories). Two crops of the same person should yield cosine similarity > 0.85. All embeddings are L2-normalised so that cosine similarity equals the dot product:

$$\hat{e} = \frac{e}{\|e\|_2}$$

2.5 InsightFace SCRFD — Face Detection

2.6 InsightFace ArcFace — Face Recognition

SCRFD localises faces in the frame; ArcFace then extracts a 512-dimensional identity vector. This vector is discriminative enough to match identities even under partial occlusion (sunglasses, masks).

Table 6: InsightFace SCRFD specifications.

Property	Value
Model file	`det_10g.onnx` (buffalo_l pack)
Architecture	SCRFD (Sample & Computation Redistribution)
Input resolution	640×640
Output	Face bounding boxes + landmarks + detection confidence

Table 7: InsightFace ArcFace specifications.

Property	Value
Model file	`w600k_r50.onnx` (buffalo_l pack)
Architecture	ResNet-50 trained with ArcFace loss
Training dataset	WebFace600K (600,000 identities)
Embedding dimension	512-D
Pack size	≈326 MB
VRAM	≈1–2 GB

2.7 GPU Distribution

```
+-----------------------------------------------+
|              cuda:0  (T4 #1)                  |
|  - RT-DETR-L              (detection)         |
|  - BotSort                (tracker)           |
|  - InsightFace SCRFD+ArcFace (faces)          |
+-----------------------------------------------+
|              cuda:1  (T4 #2)                  |
|  - YOLOv11s-Pose          (quality gate)      |
|  - CLIP-ReID              (body embeddings)   |
+-----------------------------------------------+
```

Listing 2: GPU assignment across models.

3. Cloud Infrastructure

3.1 Pinecone — Vector Database

Two separate indices are maintained for the two modalities:

Table 8: Pinecone index configuration.

Index	Dimension	Metric	Contents
`supermarket-reid`	1280	Cosine	Body prototypes (3 per person via α-FPS)
`supermarket-faces`	512	Cosine	Face embeddings (all captured faces)

Pinecone provides sub-millisecond approximate nearest-neighbour search over millions of vectors with metadata filtering.

3.2 MongoDB Atlas

MongoDB stores structured metadata per person. An example document schema is shown below:

```
{
  "global_id": 29,
  "first_seen_ts": 1746432000.0,
  "last_seen_ts":  1746432185.0,
  "faces": [
    {
      "url":       "https://ik.imagekit.io/.../face_g029_t4.jpg",
      "timestamp": 1746432185.0,
      "camera_id": "cam_01"
    }
  ],
  "sightings": [
    {
      "camera_id":          "Face_Cam_Entrance",
      "track_id":           4,
      "entry_ts":           1746432010.0,
      "exit_ts":            1746432185.0,
      "duration_sec":       175.0,
      "n_detections":       312,
      "avg_det_score":      0.87,
      "avg_skeleton_ratio": 0.72,
      "best_crop_url":      "https://ik.imagekit.io/.../body_g029_t4
          .jpg"
    }
  ]
}
```

Listing 3: Example MongoDB person document.

3.3 ImageKit — CDN

Body crops and face chips are hosted as JPEGs on ImageKit, organised by identity:

```
/supermarket_reid/
    global_id_001/
        body_g001_t2_1746432000.jpg
        face_g001_t2_1746432000.jpg
    global_id_002/
        ...
```

Listing 4: ImageKit directory structure.

4. Configuration and Thresholds

All thresholds are runtime-tunable. Default values and their roles are described below.

Table 9: Detection and quality thresholds.

Parameter	Default	Description
REID_CONF_THRESHOLD	0.75	Minimum RT-DETR confidence
POSE_CONF_THRESHOLD	0.3	Per-keypoint confidence threshold
MIN_VISIBLE_RATIO	0.5	Minimum fraction of visible keypoints

Table 10: Identity resolution thresholds.

Parameter	Default	Description
TAU	2.0	Softmax ratio threshold for confident assignment
MIN_COS_SIM	0.78	Absolute minimum cosine similarity for gallery match
MAX_IMAGES	50	Maximum crops per identity in the gallery (budget)

4.1 Detection and Quality

4.2 Identity Resolution

4.3 Face Recognition

4.4 Unknown Data Manager (DBSCAN)

4.5 Session Control

5. Module 4: Video Processing Pipeline

5.1 Gallery Hydration (Pre-Processing)

Before processing any video, the system optionally loads a previously saved `identity_buffer.pkl` file containing all body embeddings per global ID, detection confidence scores, skeleton visibility ratios, and file paths to saved crops.

```
For each gid in saved_data:
    For each entry (embedding, ratio, score, filepath):
        If filepath resolves to an actual file  -> keep it
        If filepath is broken                   -> skip this entry
    If gid has zero valid entries -> skip this gid entirely
    Otherwise                     -> load into gallery
```

Listing 5: Gallery hydration logic.

This ensures the embedding, ratio, score, and filepath arrays remain synchronised—a critical fix that prevents `IndexError` crashes during budget optimisation.

5.2 Frame Processing Loop

Each frame is processed through the following sequence of steps.

Step 1 — Person Detection

```
results = detector(frame, classes=[0], verbose=False)
```

Table 11: Face recognition thresholds.

Parameter	Default	Description
IS_FACE_CAMERA	True	Master switch for all face logic
FACE_CONF_THRESHOLD	0.7	Minimum InsightFace detection score
FACE_MATCH_THRESHOLD	0.55	Minimum cosine similarity for face match

Table 12: DBSCAN clustering parameters.

Parameter	Default	Description
UDM_EPSILON	0.15	DBSCAN distance threshold ($1 - \cos$-sim)
UDM_MIN_SIZE	5	Minimum crops required to form a cluster

Step 2 — Multi-Object Tracking

```
dets   = [xyxy, confidence, class]
tracks = tracker.update(dets, frame)
```

Step 3 — Confidence Filter

Only tracks with confidence $\geq$ REID_CONF_THRESHOLD (0.75) proceed. Lower-confidence detections are treated as noise.

Step 4 — Pose Quality Gate

```
visible_count = sum(1 for x, y, c in keypoints if c > 0.3)
ratio = visible_count / 17.0
if ratio < 0.5:  REJECT
```

Step 5 — Body Embedding

```
embeddings = extract_reid_features([crop1, crop2, ...])
# Batch CLIP-ReID, L2-normalised 1280-D, on cuda:1
```

Step 6 — Face Detection and Spatial Pairing (Face Cameras Only)

For each tracked person, the system checks whether any detected face spatially belongs to their body bounding box:

```
def get_spatially_matched_face(body_box, faces):
    bx1, by1, bx2, by2 = body_box
    for f in faces:
        face_cx = (f.bbox[0] + f.bbox[2]) / 2
        face_cy = (f.bbox[1] + f.bbox[3]) / 2
        # Face center must be inside the body box AND in the top 40%
        if (bx1 <= face_cx <= bx2
                and by1 <= face_cy <= by1 + 0.4 * (by2 - by1)):
            return f
```

Table 13: Session control parameters.

Parameter	Default	Effect
RESET_DATABASE	False	When True: wipes MongoDB, deletes local crops and pickle. **Does not** affect Pinecone or ImageKit.

```
    return None
```

Listing 6: Spatial face–body pairing.

The top-40% constraint exploits the anatomical prior that a person's face always occupies the upper portion of their body bounding box, preventing spurious matches to faces of other individuals standing behind.

Step 7 — Face Pinecone Lookup

When a face is matched to a body and the detection score > 0.7:

```
face_truth_gid = search_face_pinecone(face_embedding)
```

A cosine similarity ≥ 0.55 against the `supermarket-faces` index returns the known global ID.

Step 8 — Identity Resolution

Four resolution cases are handled:

Case A — Track already mapped to a global ID. If the GID is not yet claimed in the current frame, the observation is added to the gallery and the GID is marked as claimed. If a conflict is detected (another track has already claimed the same GID in this frame), the current track is sent to the unknown pool.

The *Duplicate Guard* (`claimed_gids_this_frame`, a `set`) ensures that two different people visible simultaneously cannot be assigned the same global ID within a single frame.

Case B — Face override (forced GID).

```
if face recognised as GID 29 BUT body was assigned GID 26:
    -> execute_identity_merge(target=29, old=26)
    -> GID 26 permanently merged into GID 29
    -> all history, embeddings, sightings migrated

if face recognised as GID 29 AND track has no body assignment:
    -> item['forced_gid'] = 29  (resolved in Case D)
```

Case C — Gallery classification (body matching).

```
best_gid, softmax_ratio, max_prob = gallery.classify_live_track(
    emb, tau=TAU, exclude_gids=claimed_gids_this_frame
)
```

The classification steps are:

1. Compute weighted centroids for each known identity (weights proportional to skeleton visibility ratio).

2. Calculate cosine similarity between the query and all centroids.
3. Exclude GIDs already claimed in this frame.
4. Apply temperature-scaled softmax (temperature = 15.0).
5. Compute softmax ratio = $P(\text{best})/P(\text{second best})$.
6. If best_sim < 0.78: reject (too dissimilar).
7. If softmax_ratio $\geq \tau = 2.0$: assign confidently.
8. Otherwise: ambiguous — forward to the Unknown Data Manager.

Case D — Unknown Data Manager (UDM). Crops that cannot be confidently assigned accumulate in the UDM pool. Every 30 frames, if the pool contains ≥ 5 items, DBSCAN clustering is performed:

```
dist_matrix = 1.0 - cosine_similarity_matrix
labels = DBSCAN(
    eps=0.15, min_samples=5, metric='precomputed'
).fit(dist_matrix).labels_
```

Listing 7: DBSCAN clustering in the UDM.

For each resulting cluster: (i) if any item carries a `forced_gid` from face recognition, the cluster inherits that identity; (ii) if the cluster centroid is close enough to an existing gallery identity (distance $\leq \varepsilon$), it is merged; (iii) otherwise a new global ID is minted. Items labelled as noise (-1) remain in the pool for the next DBSCAN run.

5.3 Self-Adaptive Gallery (`GalleryManager`)

The gallery stores up to `MAX_IMAGES` = 50 crops per identity. When a new observation pushes it over budget, the worst crop is dropped according to a combined cost function:

$$\text{drop_cost} = 0.5 \times \text{redundancy} + 0.5 \times \text{uncertainty}$$

where *redundancy* is the maximum cosine similarity to any other crop of the same person, and *uncertainty* is the normalised entropy over softmax similarities to all gallery centroids. The crop with the highest drop cost is removed: it is simultaneously redundant and confusing.

5.4 Identity Merge (`execute_identity_merge`)

When face recognition determines that two GIDs refer to the same person, the following cascade is executed:

1. **Gallery.** Move all embeddings, ratios, and scores from `old_gid` to `target_gid`.
2. **Files.** Move image crops from `global_id_026/` to `global_id_029/`.
3. **Local map.** Redirect all `track_id` $\rightarrow$ `26` mappings to `track_id` $\rightarrow$ `29`.
4. **Sighting tracker.** Update active sightings.
5. **MongoDB.** Migrate sightings array; delete the old document.

5.5 Sighting Tracking

The `SightingTracker` maintains per-track metadata: entry and exit timestamps (frame number converted to UTC via `VIDEO_START_TIME` + frame/fps), detection count, running averages of confidence and skeleton ratio, best body crop (highest `conf` × `ratio`), best face crop, and best face embedding for Pinecone upload.

When BotSort ceases reporting a track, `close_track()` is called, which finalises averages and moves the sighting to the `completed_sightings` list.

5.6 Cloud Upload (Post-Processing)

After the video finishes, all completed sightings are uploaded in parallel using 10 threads:

1. Body crop → ImageKit → URL stored in sighting.
2. Face crop → ImageKit → URL stored.
3. Face embedding → Pinecone `supermarket-faces` with metadata {global_id, face_url, camera}.
4. Sighting → MongoDB (`$push` to person array).
5. Gallery → `identity_buffer.pkl` for future hydration.

6. Module 5: Search Engine

6.1 Body Search with α-FPS Prototypes

Storing all 50 crops per person in Pinecone is inefficient. Instead, α-FPS (Adaptive Farthest Point Sampling) selects $K = 3$ prototypes that jointly maximise quality and diversity:

```
1. Pick crop with highest quality score (conf x ratio) -> Prototype
   1
2. Compute distances from every remaining crop to Prototype 1
3. For each subsequent prototype:
   combined_score = alpha * quality + (1-alpha) *
      dist_to_nearest_proto
   Pick crop with highest combined_score
4. Update distances (min of current and new prototype distance)
```

Listing 8: α-FPS prototype selection.

With $\alpha = 0.5$, quality and diversity are weighted equally. The result is three representative views (e.g. front, side, back), each of high quality. These prototypes are upserted to `supermarket-reid` with IDs such as `global_29_proto_1`.

The complete body search flow is:

```
Query image --> CLIP-ReID --> 1280-D vector
    --> Pinecone query (top_k=10)
    --> Group results by global_id (take MAX score per ID)
    --> Softmax ambiguity check (temperature=15.0)
        If ratio < tau:            "AMBIGUOUS"
        If best_score < threshold: "UNKNOWN"
        Otherwise:                 "MATCH" -> fetch MongoDB document
```

Listing 9: Body search query flow.

6.2 Face Search (Direct)

Face embeddings are highly discriminative; prototyping is unnecessary. Queries are issued directly to `supermarket-faces`.

A special handling case arises for pre-cropped face chips (retrieved from ImageKit): since the face detector requires background context, the crop is padded into a 3×-larger grey canvas before embedding:

```
def embed_face_crop(img_bgr):
    h, w = img_bgr.shape[:2]
    pad = max(h, w)
    canvas = np.full((h + 2*pad, w + 2*pad, 3), 128, dtype=np.uint8)
    canvas[pad:pad+h, pad:pad+w] = img_bgr
    return embed_face(canvas)
```

Listing 10: Pre-cropped face padding.

6.3 Movement Heatmaps

After a person is identified, the dashboard generates a spatial movement heatmap visualising all positions the matched person occupied across every camera where they appeared.

Step 1 — Collect positions. Each completed sighting stores a `best_crop_bbox` field $[x_1, y_1, x_2, y_2]$ in full-frame pixel coordinates. Centre points are derived as:

$$c_x = \frac{x_1 + x_2}{2}, \qquad c_y = \frac{y_1 + y_2}{2}$$

Sightings without a bounding box are skipped.

Step 2 — Group by camera. `build_all_camera_heatmaps()` splits the sighting list by `camera_id`; each camera receives an independent heatmap because pixel coordinates are only meaningful relative to that camera's frame dimensions.

Step 3 — Render on camera background. A reference background frame is loaded from `{CAMERA_BG_DIR}/bg_{camera_id}.png`, if available. The heatmap is overlaid on this background to show store locations directly. A plain dark canvas (640×480) is used as a fallback.

Step 4 — Gaussian density estimation. A two-pass approach avoids full-resolution Gaussian evaluation:

1. A downscaled canvas (`heat_scale` $= 0.10$) receives a circular stamp (radius $= \text{round}(\sigma \times 1.1)$) at each centre point.
2. Gaussian blur: `cv2.GaussianBlur`(k, k, σ_X).
3. Upscale to full resolution with bilinear interpolation.
4. Normalise to $[0, 255]$ and colourise with `COLORMAP_JET`.
5. Blend with background: $\alpha \cdot H + (1 - \alpha) \cdot B$.

Step 5 — Last-position marker. A filled red circle at the centre point and a cyan rectangle tracing the full bounding box mark the most recent sighting.

Table 14: Heatmap configuration parameters.

Parameter	Default	Description
`heatmap_sigma`	5	Gaussian spread radius in downscaled space
`heatmap_alpha`	0.6	Blend ratio (0.0 = background only; 1.0 = heatmap only)

Configuration. The function returns a list of `(camera_id, rgb_array, info_str)` tuples, one per camera with valid sightings. The Gradio UI renders these as a captioned gallery.

7. Module 6: Interactive Dashboard

A Gradio web interface provides three tabs:

Table 15: Dashboard tab overview.

Tab	Features
Video Processing	Upload video or enter dataset path; toggle face camera; save or discard results; reset database; adjust all thresholds via sliders; progress bar.
Person Search	Upload image or paste URL; choose body, face, or both search modes; toggle pre-cropped face flag; rebuild prototypes; displays match result, full person details, and movement heatmap per camera.
Database Browser	Two sub-tabs: Browse All (bulk card view, faces first then body sightings) and Person Detail (dropdown selector, visit history table, body and face galleries).

The application is launched with `demo.launch(share=True)` to expose a public URL on Kaggle.

8. Evaluation Results

8.1 Body Search — Brute-Force Test

Evaluation was performed against 1,317 saved body crops. Results are summarised in Table 16.

Table 16: Body search evaluation results ($n = 1{,}317$).

Metric	Value
Accuracy	85.35% (1,124 / 1,317)
Incorrect assignment	54 crops
Rejected — ambiguous	74 crops
Rejected — unknown	9 crops
Average query latency	183.8 ms
Average cosine similarity	0.925
Minimum cosine similarity	0.753

The 74 ambiguous rejections represent the safety mechanism operating correctly: the system declines to assign an identity when it is uncertain, thereby preventing false positives.

8.2 Face Search

Direct face matching via Pinecone achieves near-perfect accuracy when faces are properly detected. An average cosine similarity of 0.942 on test queries confirms the strong discriminative power of ArcFace embeddings.

9. Known Limitations and Design Decisions

9.1 Data Consistency After Reset

`RESET_DATABASE = True` clears MongoDB and local files but *does not* affect Pinecone or ImageKit. Face embeddings in Pinecone may therefore reference GIDs that no longer exist in MongoDB. A full reset requires manual deletion of Pinecone index entries and the ImageKit folder.

9.2 RAM-First Processing

All sightings accumulate in memory during video processing and are uploaded in bulk at the end. This is optimised for Kaggle and Colab environments where persistent storage is limited, but a kernel crash mid-processing will result in loss of all accumulated data.

9.3 Single-Frame Identity Lock

The `claimed_gids_this_frame` set prevents the same GID from being assigned to two tracks within a single frame. If BotSort temporarily fragments one person into two tracks, one of them is sent to the Unknown Data Manager — this is the intended safety behaviour.

9.4 Face Camera Toggle

`IS_FACE_CAMERA = False` completely disables all face logic (SCRFD, ArcFace, and Pinecone face queries). This is designed for overhead or long-range cameras where faces are never visible, saving GPU computation.

9.5 Embedding Dimension Mismatch

Body embeddings are 1280-dimensional; face embeddings are 512-dimensional. They reside in entirely separate Pinecone indices and are never compared to each other. The only link between a body record and a face record is the shared `global_id` in metadata.

9.6 Heatmap Accuracy Depends on Bounding Box Storage

The movement heatmap relies on the `best_crop_bbox` field in the MongoDB sighting record. Sightings recorded before this field was introduced to the schema will be silently skipped, causing the heatmap to under-represent early appearances of a person in mixed old/new data.

9.7 Camera Background Registration

For the heatmap overlay to be spatially meaningful, a background image must exist at {`CAMERA_BG_DIR`}/bg_{came and must exactly match the resolution of the recorded footage. If this file is missing or incorrectly sized, the heatmap falls back to a plain dark canvas and positional accuracy is lost. Best practice is to capture a background frame from each camera before deployment and save it with the correct naming convention.